
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 3

Desenvolvimento Estruturado de Programas em C

Exercícios de Autorrevisão (3.1–3.10)

Resolvidos passo a passo, com abordagem incremental
Capítulos 1 + 2 + 3

Construindo sobre os Capítulos Anteriores

No Capítulo 3, ampliamos consideravelmente nosso arsenal de ferramentas em C. Construindo sobre os fundamentos dos Capítulos 1 e 2, incorporamos agora:

- **Algoritmos e pseudocódigo** – a etapa de planejamento que precede a codificação
- **Refinamentos top-down** – a metodologia para projetar programas estruturados
- **if...else** – estrutura de seleção dupla
- **while** – nossa primeira estrutura de repetição
- **Repetição controlada por contador** (definida) e **por sentinela** (indefinida)
- **Tipo float** e cálculos com ponto flutuante
- **Operadores de atribuição compostos** (`+=`, `-=`, `*=`, `/=`, `%=`)
- **Operadores de incremento/decremento** (`++` e `--`) – prefixo e pós-fixos

Boa prática de programação

Os conceitos do Capítulo 3 são o coração da programação estruturada. Domine pseudocódigo, refinamentos top-down e **while** – esses três elementos formam a base sobre a qual todo programa não-trivial será construído.

Sumário

1	Exercício 3.1 – Preenchimento de Lacunas	2
2	Exercício 3.2 – Quatro formas de incrementar x	6
3	Exercício 3.3 – Instruções únicas em C	7
4	Exercício 3.4 – Instruções para soma	8
5	Exercício 3.5 – Soma dos inteiros de 1 a 10	9
6	Exercício 3.6 – Pós-incremento dentro de expressão	11
7	Exercício 3.7 – Instruções para potência	12
8	Exercício 3.8 – Programa: x elevado a y	12
9	Exercício 3.9 – Identificar e Corrigir Erros	14
10	Exercício 3.10 – Loop Infinito	15

1. Exercício 3.1 – Preenchimento de Lacunas

Enunciado 3.1

Preencha os espaços nas oito sentenças sobre algoritmos, controle de programa, estruturas de seleção, instruções compostas e estruturas de repetição.

Item (a) – Procedimento para resolver problema em termos de ações ordenadas

Resposta detalhada

Resposta: algoritmo

Explicação:

Da Seção 3.2: um **algoritmo** é o procedimento utilizado para resolver um problema em termos (1) das ações a serem executadas e (2) da ordem em que essas ações devem ser executadas. A ordem é fundamental – como ilustramos no exemplo do executivo “preparando-se para o trabalho”: se ele veste-se antes de tomar banho, o resultado será desastroso! Em programação, especificar a ordem correta dos passos é a essência de qualquer algoritmo.

Item (b) – Especificação da ordem de execução das instruções

Resposta detalhada

Resposta: controle do programa

Explicação:

Controle do programa é o termo técnico para a especificação da ordem em que as instruções de um programa devem ser executadas. O Capítulo 3 dedica-se quase integralmente a estudar os mecanismos de controle do programa em C: as estruturas de seleção (`if`, `if...else`) e as estruturas de repetição (`while`).

Item (c) – Os três tipos de estruturas de controle

Resposta detalhada

Resposta: sequência, seleção e repetição

Explicação:

A pesquisa de Bohm e Jacopini (1966) demonstrou um resultado notável: *qualquer programa pode ser escrito utilizando apenas três tipos de estruturas de controle*:

- a) **Sequência:** As instruções são executadas uma após a outra, na ordem em que aparecem. Esta é a estrutura “default” – está embutida na linguagem C.
- b) **Seleção:** O programa escolhe entre cursos alternativos de ação. Em C: `if` (seleção única), `if...else` (seleção dupla), `switch` (seleção múltipla,

Cap. 4).

- c) **Repetição:** Um conjunto de instruções é executado várias vezes enquanto uma condição permanece verdadeira. Em C: `while`, `do...while` e `for` (estes dois últimos no Cap. 4).

Essa descoberta é a base teórica da **programação estruturada**, que tornou quase sinônima a expressão “eliminação do `goto`”.

Item (d) – Estrutura de seleção que executa uma ação ou outra

Resposta detalhada

Resposta: `if...else`

Explicação:

A estrutura `if...else` (Seção 3.6) é chamada de *seleção dupla*: ela executa uma ação se a condição for verdadeira e outra ação diferente se a condição for falsa. Sintaxe básica:

```
1  if ( nota >= 60 ) {  
2      printf( "Aprovado\n" );  
3  } /* fim do if */  
4  else {  
5      printf( "Reprovado\n" );  
6  } /* fim do else */
```

Diferentemente do `if` simples (Cap. 2), que apenas executa ou não uma ação, o `if...else` sempre executa *exatamente um* dos dois caminhos.

Item (e) – Várias instruções agrupadas com chaves

Resposta detalhada

Resposta: **instrução composta** (ou **bloco**)

Explicação:

Um conjunto de instruções entre chaves `{` e `}` forma uma **instrução composta** (também chamada de **bloco**). Uma instrução composta pode aparecer em qualquer lugar onde uma instrução única seria válida – esse é um conceito poderoso da linguagem C.

```
1  if ( nota >= 60 ) {  
2      printf( "Aprovado\n" );           /* instrucao 1 do  
        bloco */  
3      printf( "Parabens!\n" );         /* instrucao 2 do  
        bloco */  
4      aprovados = aprovados + 1;      /* instrucao 3 do  
        bloco */  
5  } /* todas executadas se nota >= 60 */
```

Item (f) – Estrutura de repetição com condição verdadeira

Resposta detalhada

Resposta: `while`**Explicação:**

A estrutura `while` (Seção 3.7) repete uma instrução ou bloco *enquanto* uma condição permanecer verdadeira. Quando a condição se torna falsa, o laço termina e a execução continua na próxima instrução após o `while`.

```
1 contador = 1;
2 while ( contador <= 10 ) {           /* enquanto contador
   <= 10 */
3     printf( "%d\n", contador );      /* exibe o valor
   */
4     contador = contador + 1;         /* incrementa o
   contador */
5 } /* fim do while */
```

Erro comum de programação

Esquecer de alterar a variável da condição dentro do corpo do `while` é o erro mais clássico associado a essa estrutura: cria-se um **loop infinito**. Sempre garanta que algum elemento do corpo do laço modifique a condição em direção a se tornar falsa.

Item (g) – Repetição por número específico de vezes

Resposta detalhada

Resposta: **controlada por contador** (ou repetição definida)**Explicação:**

A **repetição controlada por contador** (Seção 3.8) usa uma variável-contador para especificar exatamente quantas vezes um conjunto de instruções deve ser executado. É chamada de *repetição definida* porque o número de repetições é **conhecido antes** do laço começar.

Padrão típico:

- a) Inicialize o contador (geralmente com 1)
- b) No `while`, teste se o contador atingiu o limite
- c) No corpo, execute a tarefa e incremente o contador

Item (h) – Repetição quando o número de iterações é desconhecido

Resposta detalhada

Resposta: **sentinela** (também chamado de *senalizador*, *valor artificial* ou *flag*)

Explicação:

Quando não sabemos com antecedência quantas vezes o laço será executado, usamos um **valor de sentinela** – um valor especial que o usuário digita para indicar o “fim da entrada de dados”. A repetição controlada por sentinela é também chamada de *repetição indefinida*.

Regra de ouro: O valor de sentinela deve ser escolhido de modo que *nunca* possa ser confundido com um dado válido. Por exemplo, em um programa que lê notas de teste (sempre ≥ 0), -1 é uma boa sentinela.

2. Exercício 3.2 – Quatro formas de incrementar x

Enunciado 3.2

Escreva quatro instruções diferentes em C; cada uma deve somar 1 à variável inteira `x`.

Resposta detalhada

C oferece múltiplas formas para a operação fundamental de incrementar uma variável. Cada uma delas tem o *mesmo efeito* (somar 1 a `x`) quando usada isoladamente como uma instrução:

```
1 x = x + 1;    /* forma classica usando atribuicao e
                adicao */
2 x += 1;      /* operador de atribuicao composto +=
                */
3 ++x;         /* operador de pre-incremento
                */
4 x++;         /* operador de pos-incremento
                */
```

Quando usar cada forma?

- `x = x + 1` – forma explícita, mais legível para iniciantes
- `x += 1` – forma compacta, útil para incrementos por valor diferente de 1
- `++x` ou `x++` – forma idiomática em C, especialmente em laços

Boa prática de programação

Embora as quatro formas sejam equivalentes *em uma instrução isolada*, o pré-incremento (`++x`) e o pós-incremento (`x++`) podem ter efeitos diferentes quando usados *dentro de uma expressão maior* – como veremos no Exercício 3.6.

3. Exercício 3.3 – Instruções únicas em C

Resposta detalhada

a) Atribuir $x + y$ a z e incrementar x após o cálculo:

```
1 z = x++ + y;
```

O pós-incremento $x++$ usa o valor *atual* de x na soma e *depois* incrementa x – exatamente o comportamento solicitado.

b) Multiplicar produto por 2 com $*=$:

```
1 produto *= 2;
```

Equivalente a `produto = produto * 2;`

c) Multiplicar produto por 2 com $=$ e $*$:

```
1 produto = produto * 2;
```

Forma explícita, sem o operador composto.

d) Testar se contador > 10 e exibir mensagem:

```
1 if ( contador > 10 ) {  
2     printf( "Contador e maior do que 10\n" );  
3 } /* fim do if */
```

e) Decrementar x em 1, depois subtrair de total:

```
1 total -= --x;
```

O pré-decremento $--x$ primeiro decrementa x e *depois* usa o novo valor na subtração: `total = total - (x - 1)`, com x já alterado.

f) Somar x a total, depois decrementar x :

```
1 total += x--;
```

O pós-decremento $x--$ primeiro usa o valor atual de x na soma e *depois* decrementa x .

g) Resto de q / divisor atribuído a q (duas formas):

```
1 q %= divisor;           /* forma compacta com %= */  
2 q = q % divisor;        /* forma explícita      */
```

h) Imprimir 123.4567 com 2 dígitos de precisão:

```
1 printf( "%.2f", 123.4567 );
```

Saída: 123.46 – o valor é *arredondado* (não truncado) para duas casas decimais. Como o terceiro dígito é 6 (≥ 5), arredonda-se para cima: .45 $\rightarrow$.46.

i) Imprimir 3.14159 com 3 dígitos à direita do ponto decimal:

```
1 printf( "%.3f\n", 3.14159 );
```

Saída: 3.142 – arredondado (o quarto dígito, 5, força o arredondamento para cima: .141 $\rightarrow$.142, na verdade o que acontece é que se olha o próximo dígito e arredonda-se).

4. Exercício 3.4 – Instruções para soma

Resposta detalhada

a) Declarar soma e x como int:

```
1 int soma, x;
```

b) Inicializar x em 1:

```
1 x = 1;
```

c) Inicializar soma em 0:

```
1 soma = 0;
```

d) Somar x a soma:

```
1 soma += x;           /* forma compacta */
2 soma = soma + x;     /* forma equivalente */
```

e) Imprimir mensagem com valor de soma:

```
1 printf( "A soma e: %d\n", soma );
```

Boa prática de programação

Observe a regra fundamental do Cap. 3: **contadores e totais devem ser inicializados antes do uso**. Variáveis não inicializadas contêm “lixo” – o último valor que ocupava aquela posição de memória, totalmente imprevisível.

5. Exercício 3.5 – Soma dos inteiros de 1 a 10

Enunciado 3.5

Combine as instruções do Exercício 3.4 em um programa completo que calcule a soma dos inteiros de 1 a 10, usando a estrutura `while`. O laço deve terminar quando `x` atingir 11.

Resposta detalhada

```
1  /* Calcula a soma dos inteiros de 1 a 10 */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int soma, x; /* declara variaveis soma e x */
7
8      x = 1;        /* inicializa x */
9      soma = 0;     /* inicializa soma */
10
11     while ( x <= 10 ) { /* laço enquanto x menor ou
12                         igual a 10 */
13         soma += x;      /* soma x ao total acumulado
14                         */
15         ++x;           /* incrementa x
16                         */
17     } /* fim do while */
18
19     printf( "A soma e: %d\n", soma );
20     return 0;
21 } /* fim da funcao main */
```

Saída do programa

```
A soma e: 55
```

Rastreamento (trace) das primeiras iterações:

Iteração	x (início)	soma (após soma += x)	x (após ++x)
Inicialização	1	0	—
1	1	1	2
2	2	3	3
3	3	6	4
4	4	10	5
⋮	⋮	⋮	⋮
10	10	55	11
Saída do laço	11	55	—

A condição `11 <= 10` é falsa, e o laço termina. `soma` contém a resposta correta: $1 + 2 + \dots + 10 = 55$.

6. Exercício 3.6 – Pós-incremento dentro de expressão

Enunciado 3.6

Determine os valores de `produto` e `x` após a execução de `produto *= x++;`, supondo que ambas começam com valor 5.

Resposta detalhada

Resposta: `produto = 25, x = 6`

Explicação passo a passo:

A expressão `produto *= x++;` é equivalente a `produto = produto * x++;`. O comportamento do **pós-incremento** é crítico aqui:

- `x++` primeiro **usa o valor atual de x** (que é 5) na expressão `produto * x`.
- Calcula-se `produto * 5 = 5 * 5 = 25`, que é atribuído a `produto`.
- Após** o uso do valor, `x` é incrementado para 6.

Resultado final: `produto = 25, x = 6`.

Comparação com o pré-incremento:

Se a instrução fosse `produto *= ++x;`, o resultado seria diferente:

- `++x` primeiro **incrementa x para 6**.
- Depois, calcula-se `produto * 6 = 5 * 6 = 30`.
- Resultado: `produto = 30, x = 6`.

Dica de prevenção de erro

Esta diferença sutil entre `x++` e `++x` *dentro de uma expressão* é uma fonte clássica de bugs em C. Quando estiver em dúvida sobre o comportamento, separe a operação em duas instruções para tornar o código inequívoco.

7. Exercício 3.7 – Instruções para potência

Resposta detalhada

```
a) scanf( "%d", &x );
b) scanf( "%d", &y );
c) i = 1;
d) potencia = 1;
e) potencia *= x;   (ou potencia = potencia * x;)
f) i++;   (ou ++i;, ou i = i + 1;, ou i += 1;)
g) while ( i <= y )
h) printf( "%d", potencia );
```

8. Exercício 3.8 – Programa: x elevado a y

Enunciado 3.8

Use as instruções do Exercício 3.7 para calcular x^y usando `while`.

Resposta detalhada

```
1  /* Eleva x a potencia y */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int x, y, i, potencia;  /* declara variaveis */
7
8      i = 1;                  /* inicializa contador */
9      potencia = 1;          /* inicializa potencia em 1 */
10
11     printf( "Digite a base x: " );
12     scanf( "%d", &x );
13
14     printf( "Digite o expoente y: " );
15     scanf( "%d", &y );
16
17     while ( i <= y ) {       /* enquanto i <= y
18                             */
19         potencia *= x;      /* multiplica potencia
20                             por x */
21         ++i;                /* incrementa o contador
22                             */
23     } /* fim do while */
24
25     printf( "%d elevado a %d e %d\n", x, y, potencia );
26     ;
27
28     return 0;
```

```
25 } /* fim da funcao main */
```

Saída do programa

```
Digite a base x: 2
Digite o expoente y: 10
2 elevado a 10 e 1024
```

Como funciona: multiplicamos *potencia* por *x* repetidamente, *y* vezes. Após *y* iterações, *potencia* contém x^y . Por exemplo, 2^{10} : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$.

9. Exercício 3.9 – Identificar e Corrigir Erros

Resposta detalhada

a) Código com erro:

```
1 while ( c <= 5 ) {  
2     produto *= c;  
3     ++c;
```

Erro: Falta a chave de fechamento } do while.

Correção:

```
1 while ( c <= 5 ) {  
2     produto *= c;  
3     ++c;  
4 } /* fim do while */
```

b) Código com erro:

```
1 scanf( "%.4f", &valor );
```

Erro: A precisão (.4) não é válida em uma especificação de conversão do scanf. A precisão é usada apenas em printf para controlar a formatação da saída.

Correção:

```
1 scanf( "%f", &valor );
```

c) Código com erro:

```
1 if ( sexo == 1 )  
2     printf( "Mulher\n" );  
3 se nao  
4     printf( "Homem\n" );
```

Erro: A palavra se nao (em português) não existe em C. A palavra-chave correta é else.

Correção:

```
1 if ( sexo == 1 ) {  
2     printf( "Mulher\n" );  
3 }  
4 else {  
5     printf( "Homem\n" );  
6 }
```

10. Exercício 3.10 – Loop Infinito

Enunciado 3.10

Identifique o erro no `while` a seguir (`z` começa em 100), que deveria calcular a soma dos inteiros de 100 a 1:

```
1 while ( z >= 0 )  
2     soma += z;
```

Resposta detalhada

Erro: O valor de `z` **nunca é alterado** dentro do corpo do `while`. Como `z` permanece sempre 100, a condição `z >= 0` nunca se torna falsa, e o programa entra em um **loop infinito**.

Correção: é necessário decrementar `z` dentro do laço, para que ele eventualmente chegue a `-1` e a condição se torne falsa:

```
1 while ( z >= 0 ) {  
2     soma += z;  
3     --z;          /* decrementa z para evitar loop  
                   infinito */  
4 } /* fim do while */
```

Com essa correção, `z` assume os valores 100, 99, 98, ..., 1, 0, e `soma` acumula $100 + 99 + \dots + 1 + 0 = 5050$. Quando `z` chega a `-1`, a condição falha e o laço termina.

Erro comum de programação

Loop infinito é um dos erros mais comuns em programação. A regra é simples: o corpo do laço *deve* alterar, eventualmente, alguma variável da condição em direção a torná-la falsa. Se isso não acontecer, o programa ficará rodando indefinidamente, exigindo interrupção forçada (Ctrl+C).

Gabarito Rápido – Capítulo 3: Autorrevisão

Respostas Resumidas

3.1

- a) algoritmo b) controle do programa
- c) sequência, seleção, repetição d) `if...else`
- e) instrução composta (bloco) f) `while`
- g) controlada por contador h) sentinela

3.2: `x = x + 1;` `x += 1;` `++x;` `x++;`

3.3: a) `z = x++ + y;` b) `produto *= 2;` c) `produto = produto * 2;`
d) `if (contador > 10) printf(...);` e) `total -= -x;` f) `total += x-;`
g) `q %= divisor;` ou `q = q % divisor;` h) 123.46 i) 3.142

3.6: `produto = 25, x = 6`

3.10: `z` nunca é decrementado $\Rightarrow$ loop infinito. Solução: adicionar `-z;` no corpo do laço.